

ÜBUNG 4

Syscalls, Signals, Semaphores & Shared Memory

Max Schrötter

maxschro@uni-potsdam.de

Institute for Computational Science
University of Potsdam

5. Dezember 2025



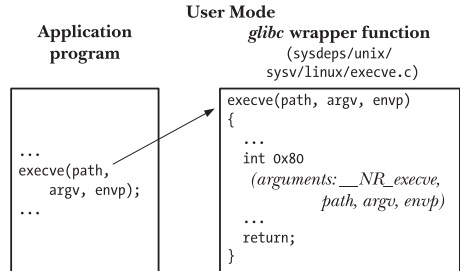
AGENDA

1. Syscalls
2. VFS & Inode
3. Hinweise Blatt 4



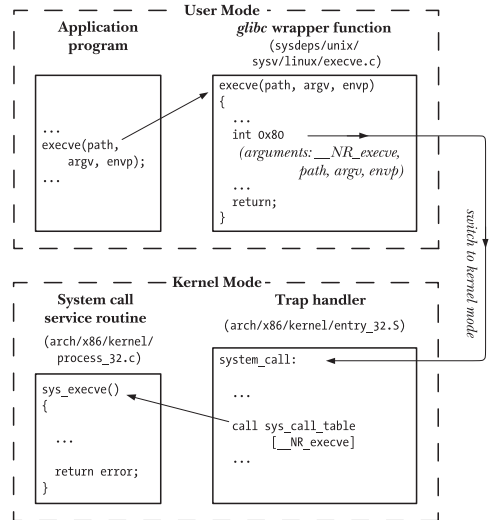
SYSCALLS - 32 BIT LINUX

- Aufrufe wie **fork()** oder **execve()** sind **Wrapperfunktionen** in der Glibc
- **Wrapperfunktionen** = Funktionen, die andere Programmkomponenten „umgeben“ (in diesem Falle aus Kernel Mode)
- aufrufendes Programm steht im Austausch mit **Wrapper**



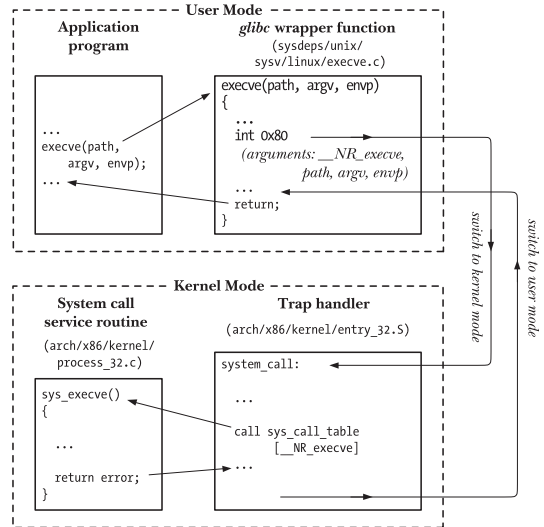
SYSCALLS - 32 BIT LINUX

- Glibc führt **Interrupt 0x80** aus um in den Kernel-Mode zu wechseln
- X86_64: **syscall** Instruction
- **Argumente** des Syscalls müssen in **Registern** gesetzt werden
- Syscalls können auch **direkt** über den **syscall()-Wrapper** aufgerufen werden
- **syscall** wird via **Nummer im ersten Argument** spezifiziert



SYSCALLS - 32 BIT LINUX

- Glibc führt **Interrupt 0x80** aus um in den Kernel-Mode zu wechseln
- X86_64: **syscall** Instruction
- **Argumente** des Syscalls müssen in **Registern** gesetzt werden
- Syscalls können auch **direkt** über den **syscall()-Wrapper** aufgerufen werden
- **syscall** wird via **Nummer im ersten Argument** spezifiziert



SYSCALL WRAPPER

Syntax:

```
long syscall(long number, ... );  
//syscall(SYS_befehl,argument_1,...);
```

```
#include <unistd.h>  
#include <stdio.h>  
#include <sys/syscall.h>
```

```
int main(int argc, char**argv){  
    int c;  
    c = syscall(SYS_getpid);  
    printf("%d", c);  
}
```

%rax	System call	%rdi	%rsi	%rdx
0	sys_read	unsigned int fd	char *buf	size_t count
1	sys_write	unsigned int fd	const char *buf	size_t count
2	sys_open	const char *filename	int flags	int mode

http://blog.rchapman.org/posts/Linux_System_Call_Table_for_x86_64/

GETTIMEOFDAY

SYNOPSIS

```
#include <sys/time.h>

int gettimeofday(struct timeval* tv, struct timezone* tz);
```

DESCRIPTION

The functions `gettimeofday()` and `settimeofday()` can get and set the time as well as a timezone.

The `tv` argument is a `struct timeval` (as specified in `<sys/time.h>`):

```
struct timeval {
    time_t      tv_sec;      /* seconds */
    suseconds_t tv_usec;     /* microseconds */
};
```

Quelle: `man gettimeofday`

HINWEISE

- Zeitmessung mit **time**
 - time ist ein Unix/Linux Programm und ein **bash-Built-In**
 - misst die Zeit, die ein Programm benötigt und schlüsselt diese nach **Userspace** und **Kernelspace** auf

```
maxschro@tiree:~$ time ./a.out  
positive
```

```
real    0m0.002s  
user    0m0.002s  
sys     0m0.000s
```

- keine Abgabe via Gitlab sondern **im PDF**.

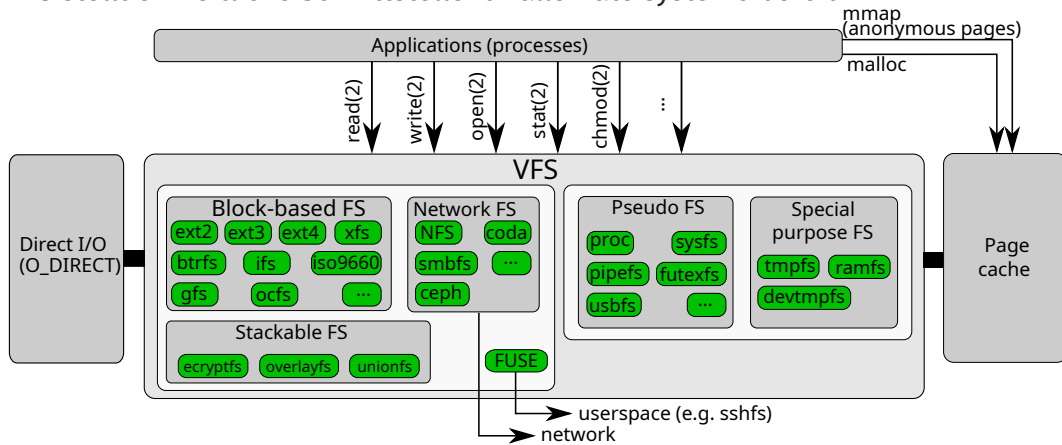
AGENDA

1. Syscalls
2. VFS & Inode
3. Hinweise Blatt 4



VIRTUAL FILE SYSTEM

- VFS stellt einheitliche Schnittstelle für alle Dateisysteme bereit



- VFS stellt Dateisysteme als Baumstruktur dar

AUFGABE 2: DAS PROC-DATEISYSTEM

Das **/proc**-Dateisystem bietet eine Schnittstelle

- zum Auslesen von Kernel-Informationen
- zum Ändern von Betriebssystemparametern (siehe **man sysctl** – `configure kernel parameters at runtime`)

/proc enthält

- Dateien und Dateiverzeichnisse mit Betriebssysteminformation wie z.B. `cpuinfo`, `interrupts`, `partitions`,...
- für jeden Prozess ein Unterverzeichnis, das nach der zugehörigen PID benannt ist
 - darin Informationen zum Prozess, z.B. **stat** (Status-Informationen), **maps** (aktuell gemappte Speicherregionen), **fd** (Datei-Deskriptoren der offenen Dateien),...

AUFGABE 2: DAS PROC-DATEISYSTEM

- Datei-Deskriptoren sind Integer, die vom System nach dem erfolgreichen Öffnen einer Datei zurückgegeben werden
- Verwendet von Syscalls wie **read**, **write**, **close**
- Neuer Prozess öffnet standardmäßig die Dateien **stdin** (Standardeingabe), **stdout** (Standardausgabe) und **stderr** (Standardfehlerausgabe)
- Anzeigen von Dateiinhalten: **cat** ist euer Freund!
- **ls -l** zeigt zusätzlich, auf welche Datei ein Symmlink verweist

AGENDA

1. Syscalls
2. VFS & Inode
3. Hinweise Blatt 4

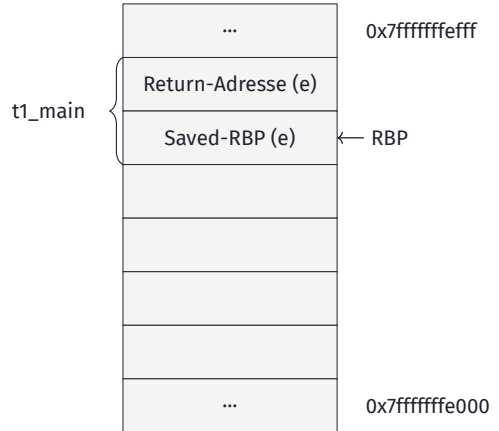


STACKFRAMES

```
void t1_f1(void){
    int x=3;
    p=&x;
    sleep(1);
}
```

```
void t1_f2(void){
    int y=5;
    k++;
    sleep(1);
}
```

```
void* t1_main(void* args){
    t1_f1();
    t1_f2();
    return NULL;
}
```

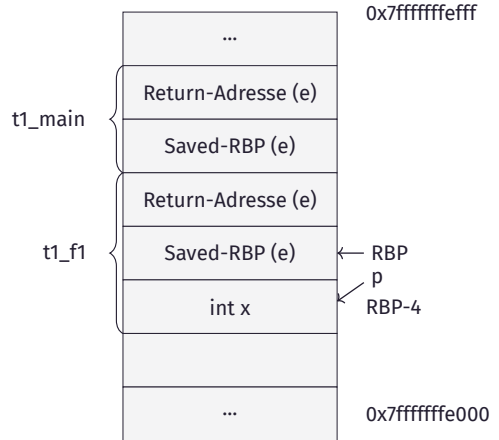


STACKFRAMES

```
void t1_f1(void){  
    int x=3;  
    p=&x;  
    sleep(1);  
}
```

```
void t1_f2(void){  
    int y=5;  
    k++;  
    sleep(1);  
}
```

```
void* t1_main(void* args){  
    t1_f1();  
    t1_f2();  
    return NULL;  
}
```

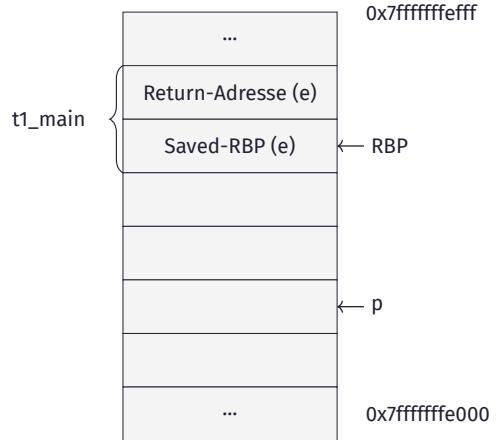


STACKFRAMES

```
void t1_f1(void){  
    int x=3;  
    p=&x;  
    sleep(1);  
}
```

```
void t1_f2(void){  
    int y=5;  
    k++;  
    sleep(1);  
}
```

```
void* t1_main(void* args){  
    t1_f1();  
    t1_f2();  
    return NULL;  
}
```

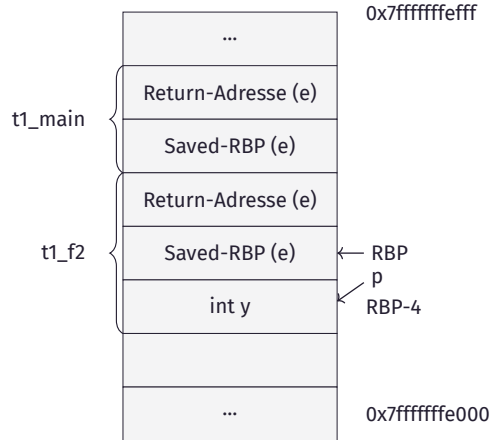


STACKFRAMES

```
void t1_f1(void){  
    int x=3;  
    p=&x;  
    sleep(1);  
}
```

```
void t1_f2(void){  
    int y=5;  
    k++;  
    sleep(1);  
}
```

```
void* t1_main(void* args){  
    t1_f1();  
    t1_f2();  
    return NULL;  
}
```



SYNCHRONISATION NEBENLÄUFIGER PROZESSE

Reader:

```
(...)  
start_read();  
read(Daten);  
end_read();  
(...)
```

Writer:

```
(...)  
start_write();  
write(Daten);  
end_write();  
(...)
```

Initialisierung gemeinsamer Variablen:

```
readcount = 0; /* Zaehler fuer Lesewuensche */  
writecount = 0; /* Zaehler fuer Schreibwuensche */  
mutex_rc = 1; /* bin. Semaphor fuer exkl. Zugriff auf readcount */  
mutex_wc = 1; /* bin. Semaphor fuer exkl. Zugriff auf writecount */  
mutex_str = 1; /* bin. Semaphor fuer exkl. Zugriff auf start_read */  
  
sem_w = 1; /* bin. Semaphor fuer exkl. Schreiben */  
sem_r = 1; /* falls r=1 dann Lesen erlaubt, falls r=0 dann Lesen nicht erlaubt  
(d. h. Schreibwunsch liegt vor) */
```

SYNCHRONISATION NEBENLÄUFIGER PROZESSE

Beispiel: zwei nebenläufige Leseprozesse R_1 und R_2 , o.B.d.A.: R_1 beginnt

R1	R2	gem. Variablen
-----	-----	-----
start_read:		

SYNCHRONISATION NEBENLÄUFIGER PROZESSE

Beispiel: zwei nebenläufige Leseprozesse R_1 und R_2 , o.B.d.A.: R_1 beginnt

R1	R2	gem. Variablen

start_read:		
DOWN(mutex_str)		mutex_str=0
DOWN(sem_r)		sem_r=0
DOWN(mutex_rc)		mutex_rc=0
-----	Zeitscheibe abgelaufen	-----

Wer darf jetzt rechnen?

SYNCHRONISATION NEBENLÄUFIGER PROZESSE

Beispiel: zwei nebenläufige Leseprozesse R_1 und R_2 , o.B.d.A.: R_1 beginnt

R1	R2	gem. Variablen

start_read:		
DOWN(mutex_str)		mutex_str=0
DOWN(sem_r)		sem_r=0
DOWN(mutex_rc)		mutex_rc=0
-----	Zeitscheibe abgelaufen	-----
	start_read:	
	DOWN(mutex_str)	mutex_str=0 (R2)

man 7 sem_overview

A semaphore is an integer whose value is **never allowed to fall below zero**. Two operations can be performed on semaphores: increment the semaphore value by one (sem_post(3)); and decrement the semaphore value by one (sem_wait(3)). If the value of a semaphore is **currently zero**, then a sem_wait(3) operation will **block** until the value becomes greater than zero.

SYNCHRONISATION NEBENLÄUFIGER PROZESSE

Beispiel: zwei nebenläufige Leseprozesse R_1 und R_2 , o.B.d.A.: R_1 beginnt

R1	R2	gem. Variablen
start_read:		
DOWN(mutex_str)		mutex_str=0
DOWN(sem_r)		sem_r=0
DOWN(mutex_rc)		mutex_rc=0
----- Zeitscheibe abgelaufen -----		
	start_read:	
	DOWN(mutex_str)	mutex_str=0 (R2)

In welchem Zustand ist R2?

SYNCHRONISATION NEBENLÄUFIGER PROZESSE

Beispiel: zwei nebenläufige Leseprozesse R_1 und R_2 , o.B.d.A.: R_1 beginnt

R1	R2	gem. Variablen
start_read:		
DOWN(mutex_str)		mutex_str=0
DOWN(sem_r)		sem_r=0
DOWN(mutex_rc)		mutex_rc=0
----- Zeitscheibe abgelaufen -----		
	start_read:	
	DOWN(mutex_str)	mutex_str=0 (R2)
----- R2 blockiert -----		

Wer darf jetzt rechnen?

SYNCHRONISATION NEBENLÄUFIGER PROZESSE

Beispiel: zwei nebenläufige Leseprozesse R_1 und R_2 , o.B.d.A.: R_1 beginnt

R1	R2	gem. Variablen
start_read:		
DOWN(mutex_str)		mutex_str=0
DOWN(sem_r)		sem_r=0
DOWN(mutex_rc)		mutex_rc=0
----- Zeitscheibe abgelaufen -----		
	start_read:	
	DOWN(mutex_str)	mutex_str=0 (R2)
	----- R2 blockiert -----	
readcount++		readcount=1
DOWN(sem_w)		sem_w=0
UP(mutex_rc)		mutex_rc=1
----- Zeitscheibe abgelaufen -----		

SYNCHRONISATION NEBENLÄUFIGER PROZESSE

Beispiel: zwei nebenläufige Leseprozesse R_1 und R_2 , o.B.d.A.: R_1 beginnt

R1	R2	gem. Variablen

start_read:		
DOWN(mutex_str)		mutex_str=0
DOWN(sem_r)		sem_r=0
DOWN(mutex_rc)		mutex_rc=0
-----	Zeitscheibe abgelaufen	-----
	start_read:	
	DOWN(mutex_str)	mutex_str=0 (R2)
	-----	-----
	R2 blockiert	
readcount++		readcount=1
DOWN(sem_w)		sem_w=0
UP(mutex_rc)		mutex_rc=1
-----	Zeitscheibe abgelaufen	-----
UP(sem_r)		sem_r=1
UP(mutex_str)		mutex_str=0 ()
-----	R2 bereit	-----

Wer darf jetzt rechnen?

SYNCHRONISATION NEBENLÄUFIGER PROZESSE

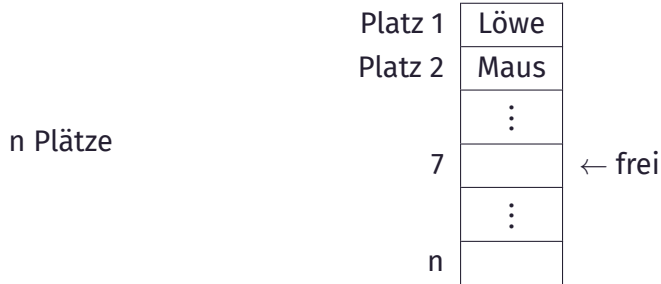
Beispiel: zwei nebenläufige Leseprozesse R_1 und R_2 , o.B.d.A.: R_1 beginnt

R1	R2	gem. Variablen
start_read:		
DOWN(mutex_str)		mutex_str=0
DOWN(sem_r)		sem_r=0
DOWN(mutex_rc)		mutex_rc=0
----- Zeitscheibe abgelaufen -----		
	start_read:	
	DOWN(mutex_str)	mutex_str=0 (R2)
----- R2 blockiert -----		
readcount++		readcount=1
DOWN(sem_w)		sem_w=0
UP(mutex_rc)		mutex_rc=1
----- Zeitscheibe abgelaufen -----		
UP(sem_r)		sem_r=1
UP(mutex_str)		mutex_str=0 ()
----- R2 bereit -----		
read(Daten)		
----- Zeitscheibe abgelaufen -----		

SYNCHRONISATION NEBENLÄUFIGER PROZESSE

		DOWN(sem_r)		sem_r=0
		DOWN(mutex_rc)		mutex_rc=0
		readcount++		readcount=2
	-----	Zeitscheibe abgelaufen	-----	
end_read:				
DOWN(mutex_rc)				mutex_rc=0 (R1)
	-----	R1 blockiert	-----	
		if(readcount==1)		(false)
		UP(mutex_rc)		mutex_rc=0 ()
	-----	R1 bereit	-----	
		UP(sem_r)		sem_r=1
	-----	Zeitscheibe abgelaufen	-----	
readcount--				readcount=1
if(readcount)				(false)
UP(mutex_rc)				mutex_rc=1
	-----	R1 fertig	-----	
		UP(mutex_str)		mutex_str=1
		read(Daten)		
		end_read:		
		...		
	-----	R2 fertig	-----	

ARCHE NOAH - GRUNDIDEE



frei := Variable, die die Nummer des nächsten zu belegenden Platzes enthält.

ARCHE NOAH - ANPASSUNG FÜR DIE AUFGABE

- **gemeinsame** Platzbuchungsliste (Array) für ein Verkehrsmittel mit n Plätzen wird zwischen **beliebig vielen** Prozessen geteilt
- **Semaphore** sollen Zugriffe auf den **gemeinsamen Speicher** (kritischer Bereich) schützen, um Race-Conditions zu verhindern

→ **Shared Memory** mit Synchronisation über **Semaphore**

→ Lösung in C:

 **man 7 shm_overview** und **man 7 sem_overview** liefern Überblick über die zu benutzenden Methoden

ARCHE NOAH

Überlegen Sie sich die folgenden Punkte:

- Welche gemeinsamen Variablen (bspw. 1 mal Semaphore und 2 mal Mutex) benötigen Sie und wie sollen Sie initialisiert werden?
- Wie sieht das Programm des Masters aus, wenn er einmal die volle Liste ausliest?
- Wie sieht das Programm der verschiedenen Worker aus?

ARCHE NOAH

Semaphore functions in C:

Two operations can be performed on semaphores: **increment the semaphore value by one** (`sem_post(3)`); and **decrement the semaphore value by one** (`sem_wait(3)`). If the value of a semaphore is currently zero, then a `sem_wait(3)` operation will block until the value becomes greater than zero.

Race condition whilst creating a semaphore?:

The `sem_open(3)` function **creates a new named semaphore or opens an existing named semaphore**. After the semaphore has been opened, it can be operated on using `sem_post(3)` and `sem_wait(3)`. When a process has finished using the semaphore, it can use `sem_close(3)` to **close the semaphore**. When all processes have finished using the semaphore, it can be **removed from the system** using `sem_unlink(3)`.

ARCHE NOAH

The POSIX shared memory API allows processes to communicate information by sharing a region of memory. [...]

- `shm_open(3)` Create and open a new object, or open an existing object. [...]
- `ftruncate(2)` Set the size of the shared memory object. (A newly created shared memory object has a length of zero.)
- `mmap(2)` Map the shared memory object into the virtual address space of the calling process.
- `munmap(2)` Unmap the shared memory object from the virtual address space of the calling process.
- `shm_unlink(3)` Remove a shared memory object name.
- `close(2)` Close the file descriptor allocated by `shm_open(3)` when it is no longer needed.

Quelle:  `man 7 shm_overview`

ARCHE NOAH

Listing: Beispielcode für POSIX-Shared-Memory

```
my_shm_struct* my_shm;  
  
// 1. Shared-Memory anlegen  
int fd = shm_open("/my_shm",  
                  O_CREAT | O_RDWR, // ggf. neu anlegen + fuer R+W  
                  S_IRUSR | S_IWUSR); // mit Zugriffsrechten: 600  
  
if (fd == -1) { /* ...Fehlerbehandlung... */ }
```

ARCHE NOAH

Listing: Beispielcode für POSIX-Shared-Memory

```
my_shm_struct* my_shm;

// 1. Shared-Memory anlegen
int fd = shm_open("/my_shm",
                  O_CREAT | O_RDWR, // ggf. neu anlegen + fuer R+W
                  S_IRUSR | S_IWUSR); // mit Zugriffsrechten: 600

if (fd == -1) { /* ...Fehlerbehandlung... */ }

// 2. Groesse festlegen
if (ftruncate(fd, sizeof(my_shm_struct)) != 0) { /* ... */ }
```

ARCHE NOAH

Listing: Beispielcode für POSIX-Shared-Memory

```
my_shm_struct* my_shm;

// 1. Shared-Memory anlegen
int fd = shm_open("/my_shm",
                  O_CREAT | O_RDWR, // ggf. neu anlegen + fuer R+W
                  S_IRUSR | S_IWUSR); // mit Zugriffsrechten: 600

if (fd == -1) { /* ...Fehlerbehandlung... */ }

// 2. Groesse festlegen
if (ftruncate(fd, sizeof(my_shm_struct)) != 0) { /* ... */ }

// 3. Speicher mappen
my_shm = mmap(NULL, // an beliebige Adresse
               sizeof(my_shm_struct), // Laenge
               PROT_READ | PROT_WRITE, MAP_SHARED, fd,
               0); // Offset
// -> Rueckgabewert: Pointer auf gemappten Speicher oder:
if (my_shm == MAP_FAILED) { /* ... */ }
```

ARCHE NOAH

Listing: Beispielcode für POSIX-Shared-Memory

```
my_shm_struct* my_shm;

// 1. Shared-Memory anlegen
int fd = shm_open("/my_shm",
                  O_CREAT | O_RDWR, // ggf. neu anlegen + fuer R+W
                  S_IRUSR | S_IWUSR); // mit Zugriffsrechten: 600

if (fd == -1) { /* ...Fehlerbehandlung... */ }

// 2. Groesse festlegen
if (ftruncate(fd, sizeof(my_shm_struct)) != 0) { /* ... */ }

// 3. Speicher mappen
my_shm = mmap(NULL, // an beliebige Adresse
               sizeof(my_shm_struct), // Laenge
               PROT_READ | PROT_WRITE, MAP_SHARED, fd,
               0); // Offset
// -> Rueckgabewert: Pointer auf gemappten Speicher oder:
if (my_shm == MAP_FAILED) { /* ... */ }

// 4. Shared-Memory verwenden
my_shm->my_integer= 100; // oder ganz anders!
```

ARCHE NOAH

Hinweise:

- Vergessen Sie nicht, den gemeinsamen Speicher **und** die Semaphore **in jedem Fall** wieder zu entfernen.
- `ls -la /dev/shm/` zeigt POSIX-Shared-Memory und -Semaphore in der Shell an
- mit `rm` lassen sich diese im Notfall wieder löschen

ARCHE NOAH

Für Aufgabenteil b):

- Signale **können** Systemaufrufe **unterbrechen**
- **normalerweise** liefert der Systemaufruf einen Fehler und setzt `errno = EINTR`

(wenn der Handler mit der Flag `SA_RESTART` registriert wurde, werden Systemaufrufe nach Unterbrechung durch ein Signal neugestartet
...das gilt jedoch **nicht für alle Systemaufrufe**.

siehe  **man 7 signal**

Folgen für Einsatz von Semaphore:

- ⚠ wenn **sem_wait** zurückkehrt, bedeutet das nicht unbedingt, dass kein weiterer Prozess mehr im kritischen Bereich ist!

Noch Fragen?

Max Schrötter

Potsdam, 5. Dezember 2025

maxschro@uni-potsdam.de